Filtrar
- iOS+
- Android
  - Fazer login com uma IU pré-criada
  - Começar
  - Gerenciar usuários
  - Autenticação com senha
  - Autenticação de link de e-mail
  - Migração de links por e-mail
  - Fazer login com o Google
  - Login do Facebook
  - Fazer login com a Apple
  - Twitter
  - GitHub
  - Microsoft
  - Yahoo
  - Login do Play Games
  - Número de telefone
  - OpenID Connect
  - Usar um sistema de autenticação personalizado
  - Autenticação anônima
  - Autenticação multifator por SMS
  - Autenticação multifator por TOTP

Firebase > Documentos > Build

Isso foi útil?

Envie comentários

Autenticar usando a Apple no Android

Nesta página
- Antes de começar
  - Participar do Programa para desenvolvedores da Apple
  - Configurar o recurso "Iniciar sessão com a Apple"
  - Ativar a Apple como um provedor de login
- Atenda os requisitos de dados anônimos da Apple
- Processar o fluxo de login com o SDK do Firebase
  - Reautenticação e vinculação de contas
- Avançado: processar o fluxo de login manualmente
- Revogação de token
- Próximas etapas

Você pode usar o SDK do Firebase para permitir que os usuários façam a autenticação com o Firebase usando o ID Apple para realizar todo o fluxo de login do OAuth 2.0.

> ★ **Importante**: para fazer login com uma conta da Apple, os usuários precisam:
>
> - ter um ID Apple com a autenticação de dois fatores (2FA) ativada;
> - fazer login no iCloud em um dispositivo Apple.
>
> Consulte Como usar o recurso Iniciar sessão com a Apple. Você também precisará atender a esses requisitos para testar sua integração com o recurso Iniciar sessão com a Apple.

Antes de começar

Para fazer login de usuários usando a Apple, primeiro configure o recurso Iniciar sessão com a Apple no site do desenvolvedor da Apple e ative-a como um provedor de login para seu projeto do Firebase.

Participar do Programa para desenvolvedores da Apple

O recurso Iniciar sessão com a Apple só pode ser configurado por membros do Programa para desenvolvedores da Apple.

Configurar o recurso "Iniciar sessão com a Apple"

No site do Desenvolvedor da Apple, faça o seguinte:

1. Associe seu site ao app conforme descrito na primeira seção de Configurar o login com a Apple para Web. Quando solicitado, registre o URL a seguir como um URL de retorno:

   ```
   https://YOUR_FIREBASE_PROJECT_ID.firebaseapp.com/__/auth/handler
   ```

   Você pode acessar o ID do projeto do Firebase na página de configurações do console do Firebase.

   Quando terminar, anote o novo ID de serviço, que será necessário na próxima seção.

2. Crie uma chave privada de Início de sessão com a Apple. Você precisará da sua nova chave privada e do ID da chave na próxima seção.
3. Se você usar qualquer um dos recursos do Firebase Authentication que enviam e-mails aos usuários, incluindo o login por link de e-mail, a verificação de endereço de e-mail, a revogação de alterações na conta, entre outros, configure o serviço de redirecionamento de e-mail privado da Apple e registre `noreply@YOUR_FIREBASE_PROJECT_ID.firebaseapp.com` (ou seu domínio de modelo de e-mail personalizado) para que a Apple possa redirecionar os e-mails enviados pelo Firebase Authentication para endereços de e-mail anônimos da Apple.

Ativar a Apple como um provedor de login

1. Adicione o Firebase ao projeto do Android. Registre a assinatura SHA-1 do app ao configurá-lo no console do Firebase.
2. No console do Firebase, abra a seção **Autenticação**. Na guia **Método de login**, ative o provedor da **Apple**. Especifique o ID do serviço que você criou na seção anterior. Além disso, na **seção de configuração do fluxo de código do OAuth**, especifique o ID de equipe da Apple, a chave privada e o ID da chave que você criou na seção anterior.

Atenda os requisitos de dados anônimos da Apple

O recurso Iniciar sessão com a Apple permite que os usuários decidam se querem que seus dados fiquem anônimos, incluindo o endereço de e-mail, ao fazer login. Os usuários que escolhem essa opção têm endereços de e-mail com o domínio `privaterelay.appleid.com`. Ao usar o recurso Iniciar sessão com a Apple no seu aplicativo, você precisa estar em conformidade com todos os termos ou políticas do desenvolvedor aplicáveis da Apple relacionados a esses IDs anônimos da Apple.

Isso inclui obter o consentimento do usuário exigido antes de associar qualquer informação pessoal de identificação direta a um ID anônimo da Apple. O uso do Firebase Authentication pode incluir as seguintes ações:

- Vincular um endereço de e-mail a um ID Apple anonimizado ou vice-versa.
- Vincular um número de telefone a um ID Apple anonimizado ou vice-versa.
- Vincular uma credencial social não anônima (Facebook, Google etc.) a um ID Apple anonimizado ou vice-versa.

Essa não é uma lista completa. Consulte o contrato de licença do programa para desenvolvedores da Apple na seção de associação da sua conta de desenvolvedor para verificar se o app atende aos requisitos da Apple.

Processar o fluxo de login com o SDK do Firebase

No Android, a maneira mais fácil de autenticar seus usuários com o Firebase usando contas da Apple é gerenciar todo o fluxo de login com o SDK do Firebase para Android.

Para processar o fluxo de login dessa forma, siga estas etapas:

1. Crie uma instância de um `OAuthProvider` usando seu Builder com o ID do provedor `apple.com`:

   Kotlin Android   Java Android

   ```
   val provider = OAuthProvider.newBuilder("apple.com")
   ```

2. **Opcional**: especifique escopos adicionais do OAuth 2.0 além do padrão que você quer solicitar ao provedor de autenticação.

   Kotlin Android   Java Android

   ```
   provider.setScopes(arrayOf("email", "name"))
   ```

   Por padrão, quando a opção **Uma conta por endereço de e-mail** é ativada, o Firebase solicita escopos de nome e e-mail. Se você mudar essa configuração para **Várias contas por endereço de e-mail**, o Firebase não solicitará nenhum escopo da Apple, a não ser que você os especifique.

3. **Opcional**: se você quiser mostrar a tela de login da Apple em um idioma diferente do inglês, configure o parâmetro `locale`. Consulte Fazer login com os documentos da Apple para ver as localidades compatíveis.

   Kotlin Android   Java Android

   ```
   // Localize the Apple authentication screen in French.
   provider.addCustomParameter("locale", "fr")
   ```

4. Use o objeto de provedor do OAuth para a autenticação no Firebase. Observe que, diferente de outras operações `FirebaseAuth`, esta assumirá o controle da sua IU abrindo uma Guia Personalizada do Chrome. Consequentemente, não faça referência à atividade em `OnSuccessListener` e `OnFailureListener` que você anexar, já que eles serão imediatamente destacados quando a operação iniciar a IU.

   Primeiro, verifique se você já recebeu uma resposta. Ao fazer login com esse método, sua Atividade fica em segundo plano, o que significa que ela pode ser recuperada pelo sistema durante o fluxo de login. Para evitar uma nova tentativa do usuário caso isso aconteça, verifique se já há um resultado disponível.

   Para verificar se há um resultado pendente, chame `getPendingAuthResult()`:

   Kotlin Android   Java Android

   ```
   val pending = auth.pendingAuthResult
   if (pending != null) {
       pending.addOnSuccessListener { authResult ->
           Log.d(TAG, "checkPending:onSuccess:$authResult")
           // Get the user profile with authResult.getUser() and
           // authResult.getAdditionalUserInfo(), and the ID
           // token from Apple with authResult.getCredential().
       }.addOnFailureListener { e ->
           Log.w(TAG, "checkPending:onFailure", e)
       }
   } else {
       Log.d(TAG, "pending: null")
   }
   ```

   Se não houver resultado pendente, inicie o fluxo de login chamando `startActivityForSignInWithProvider()`:

   Kotlin Android   Java Android

   ```
   auth.startActivityForSignInWithProvider(this, provider.build())
           .addOnSuccessListener { authResult ->
               // Sign-in successful!
               Log.d(TAG, "activitySignIn:onSuccess:${authResult.user}")
               val user = authResult.user
               // ...
           }
           .addOnFailureListener { e ->
               Log.w(TAG, "activitySignIn:onFailure", e)
           }
   ```

   Diferentemente de outros provedores compatíveis com o Firebase Auth, a Apple não fornece um URL de foto.

   Além disso, quando o usuário opta por não compartilhar e-mails com o app, a Apple fornece um endereço de e-mail exclusivo para esse usuário, no formato `xyz@privaterelay.appleid.com`, que compartilha com seu app. Se você tiver configurado o serviço de retransmissão de e-mail privado, os e-mails enviados ao endereço anônimo serão encaminhados pela Apple para o endereço de e-mail real do usuário.

   A Apple só compartilha informações do usuário, como o nome de exibição, com apps na primeira vez que um usuário faz login. Normalmente, o Firebase armazena o nome de exibição na primeira vez que um usuário faz login com a Apple, que você pode usar com `getCurrentUser().getDisplayName()`. No entanto, se você já usou a Apple para fazer login de um usuário no app sem usar o Firebase, a Apple não fornecerá o nome de exibição do usuário.

Reautenticação e vinculação de contas

É possível usar o mesmo padrão com `startActivityForReauthenticateWithProvider()`, que pode ser utilizado para recuperar uma nova credencial de operações confidenciais que exigem um login recente:

Kotlin Android   Java Android

```
// The user is already signed-in.
val firebaseUser = auth.getCurrentUser()

firebaseUser
    .startActivityForReauthenticateWithProvider(/* activity= */ this, provider.build())
    .addOnSuccessListener( authResult -> {
        // User is re-authenticated with fresh tokens and
        // should be able to perform sensitive operations
        // like account deletion and email or password
        // update.
    })
    .addOnFailureListener( e -> {
        // Handle failure.
    })
```

E você pode usar `linkWithCredential()` para vincular diferentes provedores de identidade a contas atuais.

Observe que a Apple exige que você tenha consentimento explícito dos usuários antes de vincular as contas da Apple a outros dados.

Por exemplo, para vincular uma conta do Facebook à conta atual do Firebase, use o token de acesso do login do usuário no Facebook:

Kotlin Android   Java Android

```
// Initialize a Facebook credential with a Facebook access token.
val credential = FacebookAuthProvider.getCredential(token.getToken())

// Assuming the current user is an Apple user linking a Facebook provider.
mAuth.getCurrentUser().linkWithCredential(credential)
    .addOnCompleteListener(this, task -> {
        if (task.isSuccessful()) {
          // Facebook credential is linked to the current Apple user.
          // The user can now sign in to the same account
          // with either Apple or Facebook.
        }
      });
```

Avançado: processar o fluxo de login manualmente

Também é possível autenticar com o Firebase usando uma conta da Apple. Basta processar o fluxo de login usando o SDK JS de login da Apple, criando manualmente o fluxo de OAuth ou optando por uma biblioteca OAuth, como AppAuth.

1. Para cada solicitação de login, gere uma string aleatória, um valor de uso único, que será usada para garantir que o token de ID recebido seja concedido especificamente em resposta à solicitação de autenticação do app. Essa etapa é importante para evitar ataques repetidos.

   É possível gerar um valor de uso único criptograficamente seguro no Android com `SecureRandom`, como no exemplo a seguir:

   Kotlin Android   Java Android

   ```
   private fun generateNonce(length: Int): String {
       val generator = SecureRandom()

       val charsetDecoder = StandardCharsets.US_ASCII.newDecoder()
       charsetDecoder.onUnmappableCharacter(CodingErrorAction.IGNORE)
       charsetDecoder.onMalformedInput(CodingErrorAction.IGNORE)

       val bytes = ByteArray(length)
       val inBuffer = ByteBuffer.wrap(bytes)
       val outBuffer = CharBuffer.allocate(length)
       while (outBuffer.hasRemaining()) {
           generator.nextBytes(bytes)
           inBuffer.rewind()
           charsetDecoder.reset()
           charsetDecoder.decode(inBuffer, outBuffer, false)
       }
       outBuffer.flip()
       return outBuffer.toString()
   }
   ```

   Em seguida, receba o hash SHA246 do valor de uso único como uma string hexadecimal:

   Kotlin Android   Java Android

   ```
   private fun sha256(s: String): String {
       val md = MessageDigest.getInstance("SHA-256")
       val digest = md.digest(s.toByteArray())
       val hash = StringBuilder()
       for (c in digest) {
           hash.append(String.format("%02x", c))
       }
       return hash.toString()
   }
   ```

   Você vai enviar o hash SHA256 do valor de uso único com sua solicitação de login, que a Apple não fará alterações na resposta. O Firebase valida a resposta fazendo o hash do valor de uso único original e comparando-o ao valor transmitido pela Apple.

2. Inicie o fluxo de login da Apple usando sua biblioteca OAuth ou outro método. Não se esqueça de incluir o valor de uso único com hash como parâmetro na sua solicitação.
3. Depois de receber a resposta da Apple, receba o token de ID da resposta e use-o com o valor de uso único sem hash para criar um `AuthCredential`:

   Kotlin Android   Java Android

   ```
   val credential =  OAuthProvider.newCredentialBuilder("apple.com")
       .setIdTokenWithRawNonce(appleIdToken, rawUnhashedNonce)
       .build()
   ```

4. Use a credencial do Firebase para fazer a autenticação:

   Kotlin Android   Java Android

   ```
   auth.signInWithCredential(credential)
         .addOnCompleteListener(this) { task ->
             if (task.isSuccessful) {
               // User successfully signed in with Apple ID token.
               // ...
             }
         }
   ```

Se a chamada para `signInWithCredential` for concluída, será possível usar o método `getCurrentUser` para coletar os dados da conta do usuário.

Revogação de token

A Apple exige que os apps compatíveis com a criação de contas permitam que os usuários iniciem a exclusão da conta no app, conforme descrito nas Diretrizes de avaliação da App Store.

Além disso, os apps que dão suporte ao Login com a Apple precisam usar a API REST de Login com a Apple para revogar os tokens do usuário.

Para atender a esse requisito, implemente as seguintes etapas:

1. Use o método `startActivityForSignInWithProvider()` para fazer login com a Apple e receber `AuthResult`.
2. Adquira o token de acesso para o provedor da Apple.

   Kotlin Android   Java Android

   ```
   val oauthCredential: OAuthCredential =  authResult.credential
   val accessToken = oauthCredential.accessToken
   ```

3. Revogue o token usando a API `revokeAccessToken`.

   Kotlin Android   Java Android

   ```
   mAuth.revokeAccessToken(accessToken)
     .addOnCompleteListener(this) { task ->
       if (task.isSuccessful) {
         // Access token successfully revoked
         // for the user ...
       }
   }
   ```

1. Por fim, exclua a conta de usuário e todos os dados associados a ela.

Próximas etapas

Depois que um usuário faz login pela primeira vez, uma nova conta de usuário é criada e vinculada às credenciais, que podem ser o número do telefone, o nome de usuário e a senha ou as informações do provedor de autenticação. Essa nova conta é armazenada como parte do projeto do Firebase e pode ser usada para identificar um usuário em todos os apps do projeto, seja qual for o método de login utilizado.

- Você pode receber as informações básicas de perfil do usuário do objeto `FirebaseUser` nos seus aplicativos. Consulte Gerenciar usuários.
- Nas Regras de segurança Firebase Realtime Database e Cloud Storage, você pode acessar o ID exclusivo do usuário conectado pela variável `auth` e usar essas informações para controlar quais dados um usuário pode acessar.

Os usuários podem fazer login no app usando vários provedores de autenticação. Basta vincular as credenciais desses provedores a uma conta de usuário.

Para desconectar um usuário, chame `signOut`:

Kotlin Android   Java Android

```
Firebase.auth.signOut()
```

MainActivity.kt

Isso foi útil?

Envie comentários

Exceto em caso de indicação contrária, o conteúdo desta página é licenciado de acordo com a Licença de atribuição 4.0 do Creative Commons, e as amostras de código são licenciadas de acordo com a Licença Apache 2.0. Para mais detalhes, consulte as políticas do site do Google Developers. Java é uma marca registrada da Oracle e/ou afiliadas.

Última atualização 2026-05-09 UTC.

Saiba mais
Guias do desenvolvedor
Referência da API e do SDK
Amostras
Bibliotecas
GitHub

Permanecer conectado
Confira o blog
Find us on Reddit
Follow on X
Subscribe on YouTube
Participar de um evento

Suporte
Entrar em contato com o suporte
Stack Overflow
Google group
Notas da versão
Diretrizes da marca
Perguntas frequentes

Android Chrome Firebase Google Cloud Platform Todos os produtos

Termos de Serviço | Privacidade Português –…